Gold Price Analysis and Forecasting (2016-2026)

This project presents a comprehensive time series analysis and forecasting study of global gold prices using a 10-year historical dataset. Gold plays a central role in global financial markets as a store of value, a hedge against inflation, and a safe-haven asset during economic uncertainty. Because of its sensitivity to macroeconomic indicators, geopolitical events, currency fluctuations, and interest rate movements, analyzing gold price behavior provides valuable insights into broader economic conditions.

The objective of this project is to examine historical gold price movements, identify long-term trends and volatility patterns, and develop predictive models capable of forecasting future price behavior. By applying statistical techniques and machine learning methods, the study aims to understand underlying time series characteristics such as seasonality, trend dynamics, and price fluctuations.

This analysis is designed to support investors, financial analysts, economists, and data scientists in making data-driven decisions by transforming historical price data into actionable insights through advanced time series modeling and forecasting techniques.

Business Problem Statement

Gold is widely used as a strategic investment asset and a hedge against inflation, currency depreciation, and economic instability. However, gold prices are highly volatile and influenced by multiple macroeconomic and geopolitical factors, making it difficult for investors and financial institutions to accurately predict future price movements.

The core business problem is the lack of reliable, data-driven forecasting mechanisms that can help stakeholders anticipate gold price trends and volatility. Without accurate forecasting, investors face increased risk exposure, portfolio misallocation, and suboptimal decision-making.

This project aims to address that challenge by analyzing historical gold price data and developing predictive time series models capable of forecasting future prices. The goal is to enable informed investment strategies, improve risk management, and support financial planning through robust statistical and machine learning-based forecasting techniques.

In [3]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import plotly.express as px
import plotly.graph_objects as go
from plotly.subplots import make_subplots
from sklearn.model_selection import train_test_split, TimeSeriesSplit
from sklearn.preprocessing import StandardScaler, MinMaxScaler
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
from sklearn.ensemble import RandomForestRegressor, GradientBoostingRegressor
from sklearn.linear_model import LinearRegression, Ridge
from statsmodels.tsa.stattools import adfuller, acf, pacf
from statsmodels.tsa.seasonal import seasonal_decompose
from statsmodels.tsa.arima.model import ARIMA
from statsmodels.tsa.statespace.sarimax import SARIMAX
from prophet import Prophet
import warnings
```

```
warnings.filterwarnings('ignore')

print("✅ All libraries imported successfully!")
print(f"📊 Analysis Date: {pd.Timestamp.now().strftime('%Y-%m-%d')}")
```

```
✅ All libraries imported successfully!
📊 Analysis Date: 2026-02-14
```

In [4]:
```
gd = pd.read_csv(r"C:\Users\papus\OneDrive\Documents\Data set .csv\gold_price_10y.csv")
print("Data set Loaded Succesfully")
```

```
Data set Loaded Succesfully
```

In [5]:
```
gd.head()
```

Out[5]:

	Date	Close	High	Low	Open	Volume	Adj Close	Daily_Return	MA_20	MA_50	MA_200	Volatility_20	Year	Month	Day_of_Week	Quarter
0	29-01-2016	106.949997	107.000000	106.260002	106.610001	8098700	106.949997	NaN	NaN	NaN	NaN	NaN	2016	1	4	1
1	01-02-2016	108.050003	108.150002	107.529999	107.540001	10471800	108.050003	1.028524	NaN	NaN	NaN	NaN	2016	2	0	1
2	02-02-2016	108.089996	108.180000	107.349998	107.919998	6656000	108.089996	0.037014	NaN	NaN	NaN	NaN	2016	2	1	1
3	03-02-2016	109.250000	109.580002	107.900002	107.910004	15785200	109.250000	1.073183	NaN	NaN	NaN	NaN	2016	2	2	1
4	04-02-2016	110.570000	110.699997	109.919998	110.449997	13213700	110.570000	1.208238	NaN	NaN	NaN	NaN	2016	2	3	1

In [6]:
```
gd.columns
```

Out[6]:
```
Index(['Date', 'Close', 'High', 'Low', 'Open', 'Volume', 'Adj Close',
       'Daily_Return', 'MA_20', 'MA_50', 'MA_200', 'Volatility_20', 'Year',
       'Month', 'Day_of_Week', 'Quarter'],
      dtype='object')
```

In [7]:
```
gd.dtypes
```

Out[7]:
```
Date              object
Close            float64
High             float64
Low              float64
Open             float64
Volume             int64
Adj Close        float64
Daily_Return     float64
MA_20            float64
MA_50            float64
MA_200           float64
Volatility_20    float64
Year               int64
Month              int64
Day_of_Week        int64
Quarter            int64
dtype: object
```

In [8]:
```
gd['Date'] = pd.to_datetime(gd['Date'], format='%d-%m-%Y')
```

In [10]:
```
# Sorting dataset by Date
gd = gd.sort_values(by='Date')
```

In [11]:
```
# Setting Date as index
gd.set_index('Date', inplace=True)
```

In [12]:
```
numeric_cols = gd.columns
gd[numeric_cols] = gd[numeric_cols].apply(pd.to_numeric, errors='coerce')
```

In [13]:
```
gd.isnull().sum()
```

Out[13]:
```
Close              0
High               0
Low                0
Open               0
Volume             0
Adj Close          0
Daily_Return       1
MA_20             19
MA_50             49
MA_200           199
Volatility_20     20
Year               0
Month              0
Day_of_Week        0
Quarter            0
dtype: int64
```

In [14]:
```
gd.duplicated().sum()
```

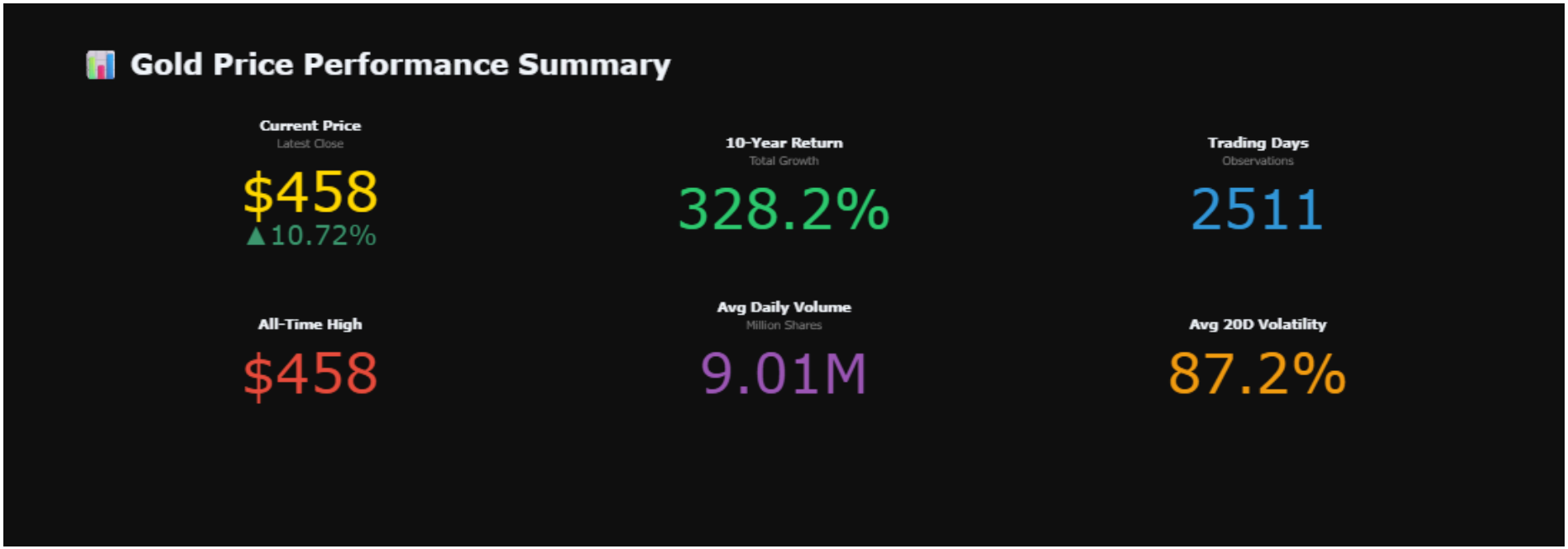
Out[14]: np.int64(0)

Exploratory Data Analysis

Visual Summary

In [15]:
```
# ==============================
# GOLD KPI DASHBOARD
# ==============================

# Calculate key metrics safely
current_price = gd['Close'].iloc[-1]
start_price = gd['Close'].iloc[0]
total_return = ((current_price / start_price) - 1) * 100
avg_daily_volume = gd['Volume'].mean()
max_price = gd['Close'].max()
avg_volatility = gd['Volatility_20'].mean()

# Safe 20-day delta reference
delta_reference = gd['Close'].iloc[-21] if len(gd) > 21 else gd['Close'].iloc[0]

# Create subplot
fig = make_subplots(
    rows=2, cols=3,
    specs=[[{"type": "indicator"}, {"type": "indicator"}, {"type": "indicator"}],
           [{"type": "indicator"}, {"type": "indicator"}, {"type": "indicator"}]]
)

# Row 1
fig.add_trace(go.Indicator(
    mode="number+delta",
    value=current_price,
    title={"text": "<b>Current Price</b><br><span style='font-size:0.8em;color:gray'>Latest Close</span>"},
    number={"prefix": "$", "font": {"size": 38, "color": "#FFD700"}},
    delta={"reference": delta_reference, "relative": True, "valueformat": ".2%"}
), row=1, col=1)

fig.add_trace(go.Indicator(
    mode="number",
    value=round(total_return, 2),
    title={"text": "<b>10-Year Return</b><br><span style='font-size:0.8em;color:gray'>Total Growth</span>"},
    number={"suffix": "%", "font": {"size": 38, "color": "#2ecc71"}}
), row=1, col=2)

fig.add_trace(go.Indicator(
    mode="number",
    value=len(gd),
    title={"text": "<b>Trading Days</b><br><span style='font-size:0.8em;color:gray'>Observations</span>"},
    number={"font": {"size": 38, "color": "#3498db"}}
```

```
), row=1, col=3)

# Row 2
fig.add_trace(go.Indicator(
    mode="number",
    value=round(max_price, 2),
    title={"text": "<b>All-Time High</b>"},
    number={"prefix": "$", "font": {"size": 38, "color": "#e74c3c"}}
), row=2, col=1)

fig.add_trace(go.Indicator(
    mode="number",
    value=round(avg_daily_volume / 1e6, 2),
    title={"text": "<b>Avg Daily Volume</b><br><span style='font-size:0.8em;color:gray'>Million Shares</span>"},
    number={"suffix": "M", "font": {"size": 38, "color": "#9b59b6"}}
), row=2, col=2)

fig.add_trace(go.Indicator(
    mode="number",
    value=round(avg_volatility * 100, 2),
    title={"text": "<b>Avg 20D Volatility</b>"},
    number={"suffix": "%", "font": {"size": 38, "color": "#f39c12"}}
), row=2, col=3)

fig.update_layout(
    height=380,
    template='plotly_dark',
    title_text='<b>📊 Gold Price Performance Summary</b>',
    title_font_size=20
)

fig.show()
```



Price Trend over 10 years

In [16]:

```
# ============================
# 10-Year Gold Price Trend
# ============================

fig = go.Figure()

# Close Price
fig.add_trace(go.Scatter(
    x=gd.index,
    y=gd['Close'],
    mode='lines',
    name='Close Price',
    line=dict(color='#FFD700', width=2)
))

# Moving Averages
fig.add_trace(go.Scatter(
    x=gd.index,
    y=gd['MA_20'],
    mode='lines',
    name='MA 20',
    line=dict(color='#3498db', width=1)
))

fig.add_trace(go.Scatter(
```

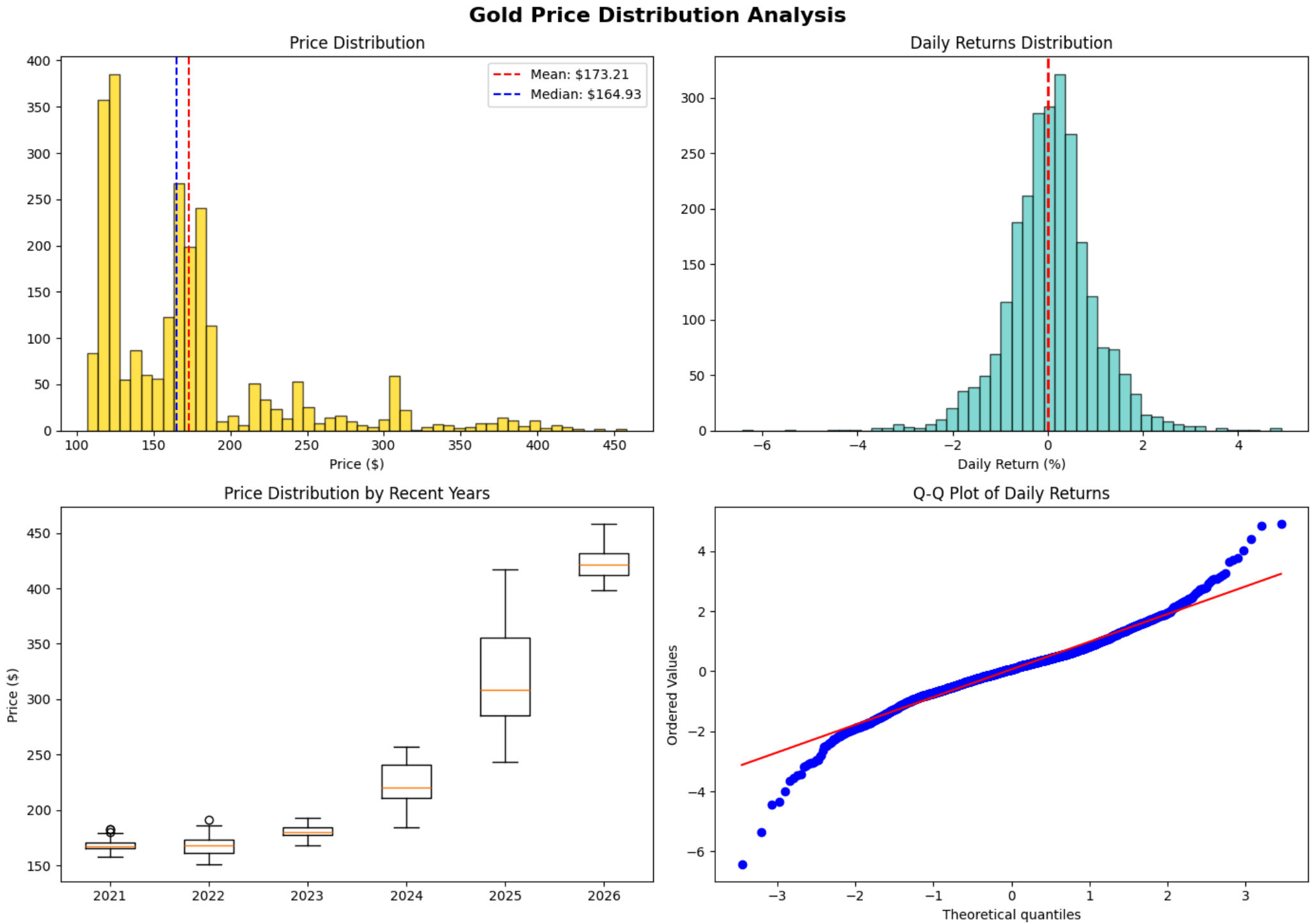
```
    x=gd.index,
    y=gd['MA_50'],
    mode='lines',
    name='MA 50',
    line=dict(color='#2ecc71', width=1)
))

fig.add_trace(go.Scatter(
    x=gd.index,
    y=gd['MA_200'],
    mode='lines',
    name='MA 200',
    line=dict(color='#e74c3c', width=1.5)
))

# Layout
fig.update_layout(
    template="plotly_dark",
    title="<b>📈 Gold Price Trend (10 Years)</b>",
    xaxis_title="Date",
    yaxis_title="Price (USD)",
    height=600,
    hovermode="x unified"
)

# Add Range Selector
fig.update_xaxes(
    rangeslider_visible=True,
    rangeselector=dict(
        buttons=list([
            dict(count=1, label="1M", step="month", stepmode="backward"),
            dict(count=6, label="6M", step="month", stepmode="backward"),
            dict(count=1, label="1Y", step="year", stepmode="backward"),
            dict(count=5, label="5Y", step="year", stepmode="backward"),
            dict(step="all", label="All")
        ])
    )
)

fig.show()
```

Price Distribution Analysis

In [18]:

```
# ============================
# Price Distribution Analysis
# ============================

from scipy import stats

fig, axes = plt.subplots(2, 2, figsize=(14, 10))
fig.suptitle('Gold Price Distribution Analysis', fontsize=16, fontweight='bold')

# 1 Histogram (Close Price)
axes[0, 0].hist(gd['Close'], bins=50, color='#FFD700', edgecolor='black', alpha=0.7)
axes[0, 0].axvline(gd['Close'].mean(), color='red', linestyle='--',
```

```python
                    label=f'Mean: ${gd["Close"].mean():.2f}')
axes[0, 0].axvline(gd['Close'].median(), color='blue', linestyle='--',
                    label=f'Median: ${gd["Close"].median():.2f}')
axes[0, 0].set_title('Price Distribution')
axes[0, 0].set_xlabel('Price ($)')
axes[0, 0].legend()

# 2️⃣ Daily Returns Distribution
axes[0, 1].hist(gd['Daily_Return'].dropna(), bins=50,
                color='#4ECDC4', edgecolor='black', alpha=0.7)
axes[0, 1].axvline(0, color='red', linestyle='--', linewidth=2)
axes[0, 1].set_title('Daily Returns Distribution')
axes[0, 1].set_xlabel('Daily Return (%)')

# 3️⃣ Boxplot by Year (Dynamic)
recent_years = sorted(gd['Year'].unique())[-6:]   # last 6 years automatically
axes[1, 0].boxplot(
    [gd[gd['Year'] == y]['Close'].values for y in recent_years],
    labels=[str(y) for y in recent_years]
)
axes[1, 0].set_title('Price Distribution by Recent Years')
axes[1, 0].set_ylabel('Price ($)')

# 4️⃣ Q-Q Plot (Normality Check for Returns)
stats.probplot(gd['Daily_Return'].dropna(), dist="norm", plot=axes[1, 1])
axes[1, 1].set_title('Q-Q Plot of Daily Returns')

plt.tight_layout()
plt.show()

# ==============================
# Descriptive Statistics
# ==============================

print("\n📊 DESCRIPTIVE STATISTICS")
print("=" * 50)
print(gd[['Open', 'High', 'Low', 'Close', 'Volume', 'Daily_Return']].describe().round(2))
```


Gold Price Distribution Analysis
Price Distribution
Mean: $173.21
Median: $164.93
Price ($)
Daily Returns Distribution
Daily Return (%)
Price Distribution by Recent Years
Price ($)
2021
2022
2023
2024
2025
2026
Q-Q Plot of Daily Returns
Ordered Values
Theoretical quantiles

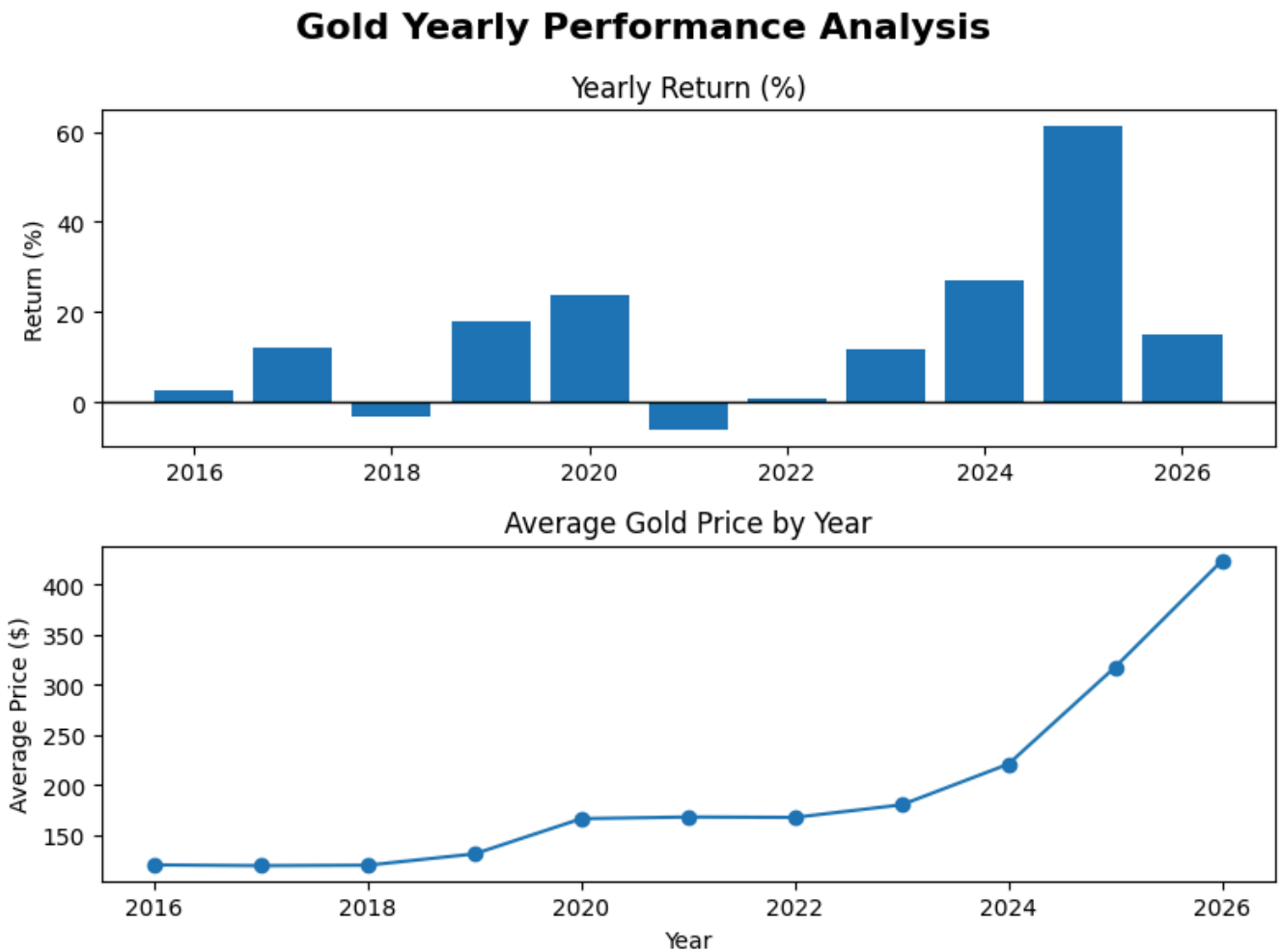
```
📊 DESCRIPTIVE STATISTICS
==================================================
          Open     High      Low    Close       Volume  Daily_Return
count  2511.00  2511.00  2511.00  2511.00      2511.00       2510.00
mean    173.20   173.94   172.40   173.21   9006662.80          0.06
std      63.13    63.50    62.68    63.17   5063949.63          0.94
min     106.61   107.00   106.26   106.95   1436500.00         -6.43
25%     123.70   123.99   123.20   123.65   5779900.00         -0.45
50%     165.02   165.72   164.29   164.93   7757000.00          0.07
75%     183.40   184.03   182.26   183.40  10696100.00          0.55
max     454.11   458.75   453.45   458.00  62025000.00          4.90
```

Yearly Performance Analysis

In [22]:

```python
# ==============================
# Yearly Performance Analysis
# ==============================

# Calculate yearly metrics
yearly_data = gd.groupby('Year').agg({
    'Close': ['first', 'last', 'mean', 'std'],
    'Daily_Return': 'std'
})

yearly_data.columns = ['Start_Price', 'End_Price', 'Avg_Price',
                       'Price_STD', 'Return_Volatility']

# Calculate yearly return
yearly_data['Yearly_Return_%'] = (
    (yearly_data['End_Price'] / yearly_data['Start_Price']) - 1
) * 100

yearly_data = yearly_data.reset_index()

# ==============================
# Visualization
# ==============================

fig, axes = plt.subplots(2, 1, figsize=(8, 6))
fig.suptitle("Gold Yearly Performance Analysis", fontsize=16, fontweight='bold')

# 1️⃣ Yearly Return
axes[0].bar(yearly_data['Year'], yearly_data['Yearly_Return_%'])
axes[0].axhline(0, color='black', linewidth=1)
axes[0].set_title("Yearly Return (%)")
axes[0].set_ylabel("Return (%)")

# 2️⃣ Average Price Trend by Year
axes[1].plot(yearly_data['Year'], yearly_data['Avg_Price'], marker='o')
axes[1].set_title("Average Gold Price by Year")
axes[1].set_ylabel("Average Price ($)")
```

```python
axes[1].set_xlabel("Year")

plt.tight_layout()
plt.show()

# ==============================
# Summary Insights
# ==============================

best_year = yearly_data.loc[yearly_data['Yearly_Return_%'].idxmax()]
worst_year = yearly_data.loc[yearly_data['Yearly_Return_%'].idxmin()]

print("\n📊 YEARLY SUMMARY")
print("=" * 50)
print(f"Best Year: {int(best_year['Year'])} "
      f"({best_year['Yearly_Return_%']:.2f}%)")

print(f"Worst Year: {int(worst_year['Year'])} "
      f"({worst_year['Yearly_Return_%']:.2f}%)")
```




```
📊 YEARLY SUMMARY
==================================================
Best Year: 2025 (61.48%)
Worst Year: 2021 (-6.24%)
```

In [21]:

```python
# ============================
# Yearly Performance in a tabular format
# ============================

yearly_perf = gd.groupby('Year').agg({
    'Close': ['first', 'last', 'mean', 'min', 'max', 'std'],
    'Volume': 'mean',
    'Daily_Return': 'std'
}).round(2)

yearly_perf.columns = [
    'Open_Price', 'Close_Price', 'Avg_Price',
```

```python
    'Min_Price', 'Max_Price', 'Price_Std',
    'Avg_Volume', 'Volatility'
]

# Annual Return %
yearly_perf['Annual_Return_%'] = (
    (yearly_perf['Close_Price'] / yearly_perf['Open_Price']) - 1
) * 100

# Price Range
yearly_perf['Price_Range'] = (
    yearly_perf['Max_Price'] - yearly_perf['Min_Price']
)

yearly_perf = yearly_perf.round(2)

print("📅 YEARLY PERFORMANCE SUMMARY")
print("=" * 80)

display(
    yearly_perf.style
        .background_gradient(cmap='YlOrRd', subset=['Annual_Return_%'])
        .format({
            'Annual_Return_%': '{:.1f}%',
            'Avg_Volume': '{:,.0f}'
        })
)
```

```
📅 YEARLY PERFORMANCE SUMMARY
================================================================================
```

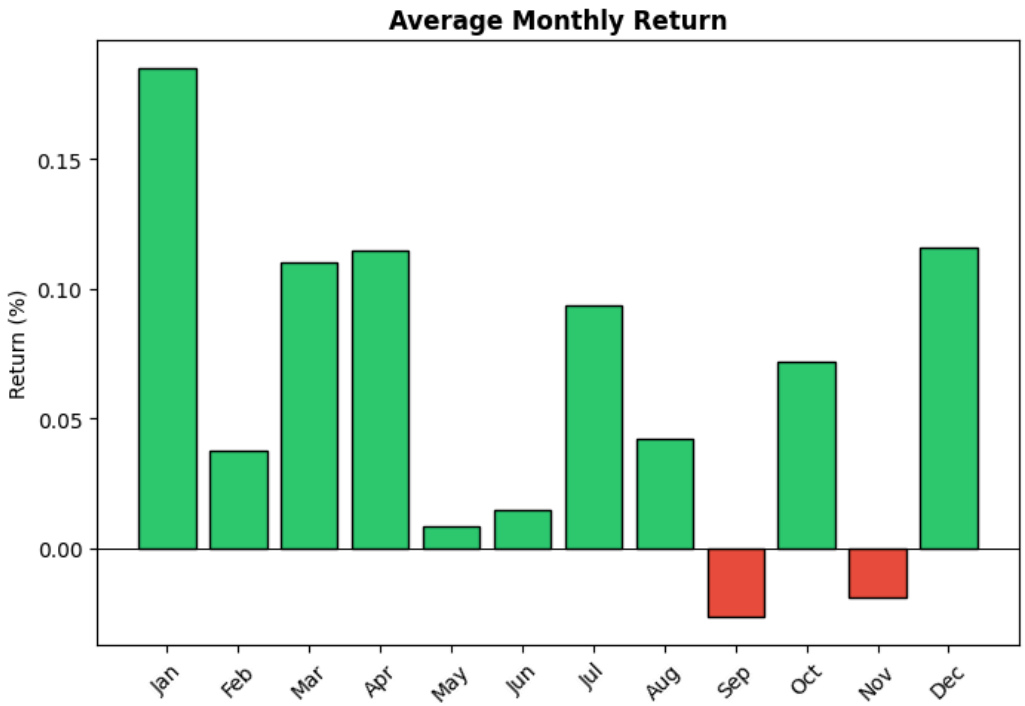
Year	Open_Price	Close_Price	Avg_Price	Min_Price	Max_Price	Price_Std	Avg_Volume	Volatility	Annual_Return_%	Price_Range
2016	106.950000	109.610000	120.460000	106.950000	130.520000	5.810000	11,078,062	1.030000	2.5%	23.570000
2017	110.470000	123.650000	119.710000	110.470000	128.130000	3.130000	7,404,134	0.620000	11.9%	17.660000
2018	125.150000	121.250000	120.180000	111.100000	128.830000	5.230000	7,545,830	0.610000	-3.1%	17.730000
2019	121.330000	142.900000	131.520000	119.940000	146.660000	8.620000	8,723,756	0.730000	17.8%	26.720000
2020	143.950000	178.360000	166.650000	138.040000	193.890000	13.110000	12,298,005	1.230000	23.9%	55.850000
2021	182.330000	170.960000	168.310000	157.490000	182.870000	4.620000	8,240,221	0.860000	-6.2%	25.380000
2022	168.330000	169.640000	167.910000	151.230000	191.510000	8.780000	8,322,585	0.960000	0.8%	40.280000
2023	171.060000	191.170000	180.450000	168.350000	192.590000	5.490000	7,303,350	0.840000	11.8%	24.240000
2024	190.720000	242.130000	221.100000	184.420000	257.500000	20.420000	6,909,356	0.950000	27.0%	73.080000
2025	245.420000	396.310000	317.910000	243.190000	416.740000	44.290000	11,872,115	1.250000	61.5%	173.550000
2026	398.280000	458.000000	424.030000	398.280000	458.000000	16.870000	17,380,853	1.270000	15.0%	59.720000

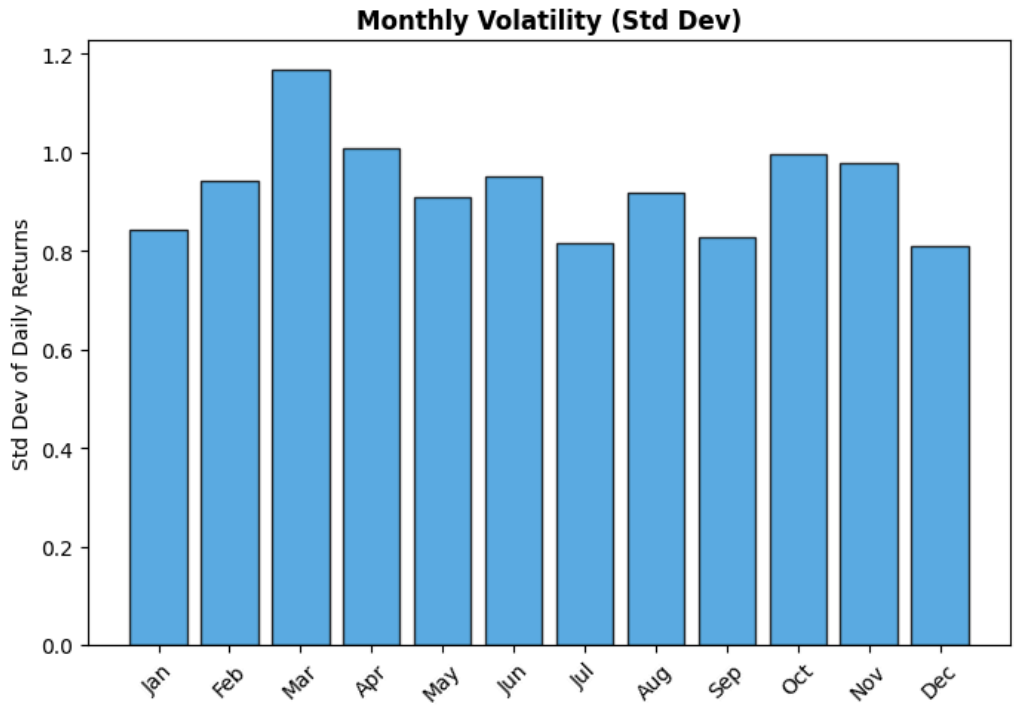
Seasonality Analysis

In [23]:

```
# ==============================
# Monthly Seasonality Analysis
# ==============================

# Monthly average return (no annualization assumption)
monthly_avg = gd.groupby('Month')['Daily_Return'].mean()
monthly_std = gd.groupby('Month')['Daily_Return'].std()

fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# Month labels
months = ['Jan', 'Feb', 'Mar', 'Apr', 'May', 'Jun',
          'Jul', 'Aug', 'Sep', 'Oct', 'Nov', 'Dec']

# Align values properly
monthly_avg = monthly_avg.reindex(range(1, 13))
monthly_std = monthly_std.reindex(range(1, 13))

# Color logic for returns
colors = ['#2ecc71' if x > 0 else '#e74c3c' for x in monthly_avg]

# 1️⃣ Monthly Returns
axes[0].bar(months, monthly_avg, color=colors, edgecolor='black')
axes[0].axhline(0, color='black', linewidth=0.7)
axes[0].set_title('Average Monthly Return', fontsize=12, fontweight='bold')
```

```python
axes[0].set_ylabel('Return (%)')
axes[0].tick_params(axis='x', rotation=45)

# 2️⃣ Monthly Volatility
axes[1].bar(months, monthly_std, color='#3498db', edgecolor='black', alpha=0.8)
axes[1].set_title('Monthly Volatility (Std Dev)', fontsize=12, fontweight='bold')
axes[1].set_ylabel('Std Dev of Daily Returns')
axes[1].tick_params(axis='x', rotation=45)

plt.tight_layout()
plt.show()


# ============================
# Day of Week Analysis
# ============================

dow_names = ['Monday', 'Tuesday', 'Wednesday', 'Thursday', 'Friday']
dow_returns = gd.groupby('Day_of_Week')['Daily_Return'].mean()

print("\n📅 AVERAGE DAILY RETURNS BY DAY OF WEEK")
print("=" * 50)

for i, day in enumerate(dow_names):
    if i in dow_returns.index:
        print(f"{day}: {dow_returns[i]:.4f}%")
```





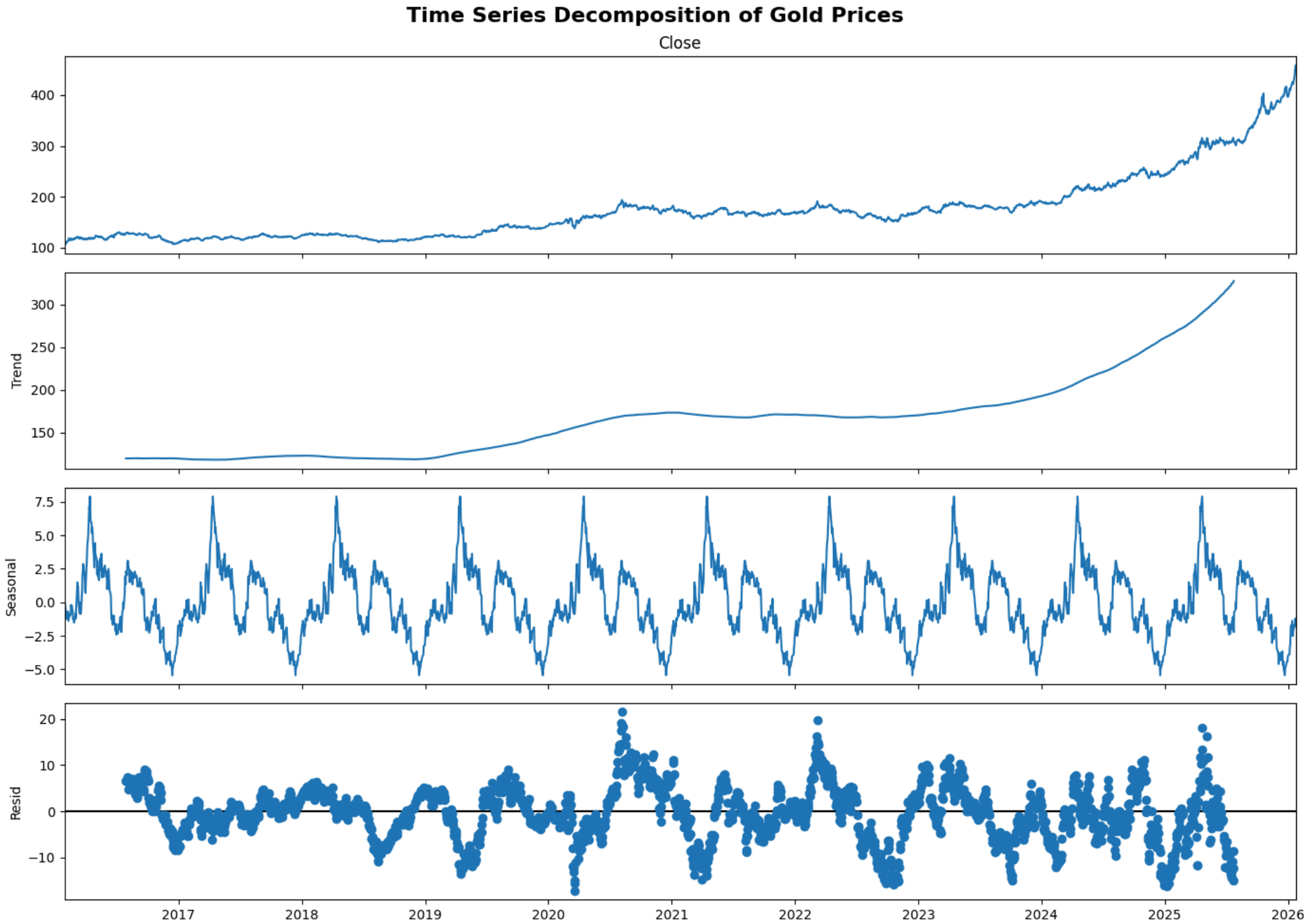
```
📅 AVERAGE DAILY RETURNS BY DAY OF WEEK
==================================================
Monday: 0.0491%
Tuesday: 0.0647%
Wednesday: 0.0570%
Thursday: 0.0753%
Friday: 0.0648%
```

📊 Statistical Analysis

Time- Series Decomposition

In [24]:

```python
# ============================
# Time Series Decomposition
# ============================

from statsmodels.tsa.seasonal import seasonal_decompose

# Use only Close price
ts = gd['Close']

# Decomposition (additive model)
decomposition = seasonal_decompose(
    ts,
    model='additive',
    period=252   # Approx yearly seasonality (trading days)
)

# Plot components
fig = decomposition.plot()
fig.set_size_inches(14, 10)

plt.suptitle("Time Series Decomposition of Gold Prices",
             fontsize=16, fontweight='bold')

plt.tight_layout()
plt.show()
```


Time Series Decomposition of Gold Prices
Close
400
300
200
100
Trend
300
250
200
150
Seasonal
7.5
5.0
2.5
0.0
−2.5
−5.0
Resid
20
10
0
−10
2017
2018
2019
2020
2021
2022
2023
2024
2025
2026

Seasonality Test

Augmented Dickey-Fuller (ADF) Test

In [25]:

```
# ==============================
# ADF Test Function
# ==============================

def adf_test(series, name):
    result = adfuller(series.dropna())

    print(f"\n📊 ADF Test for {name}")
    print("=" * 40)
    print(f"ADF Statistic: {result[0]:.4f}")
    print(f"p-value: {result[1]:.6f}")
    print("Critical Values:")

    for key, value in result[4].items():
        print(f"   {key}: {value:.4f}")

    if result[1] < 0.05:
        print("➜ Stationary (Reject H0 at 5% level)")
        return True
    else:
        print("➜ Non-Stationary (Fail to Reject H0)")
        return False


# ==============================
# 1️⃣ Test Original Price Series
# ==============================

is_stationary_price = adf_test(gd['Close'], 'Gold Price')


# ==============================
# 2️⃣ Test Daily Returns
# ==============================

is_stationary_returns = adf_test(gd['Daily_Return'], 'Daily Returns')


# ==============================
# 3️⃣ First Difference of Price
# ==============================

gd['Price_Diff'] = gd['Close'].diff()

is_stationary_diff = adf_test(gd['Price_Diff'], 'First Difference of Price')
```

```
📊 ADF Test for Gold Price
========================================
ADF Statistic: 5.7627
p-value: 1.000000
Critical Values:
   1%: -3.4330
   5%: -2.8627
   10%: -2.5674
➜ Non-Stationary (Fail to Reject H0)

📊 ADF Test for Daily Returns
========================================
ADF Statistic: -49.7326
p-value: 0.000000
Critical Values:
   1%: -3.4330
   5%: -2.8627
   10%: -2.5674
➜ Stationary (Reject H0 at 5% level)

📊 ADF Test for First Difference of Price
========================================
ADF Statistic: -6.2293
p-value: 0.000000
Critical Values:
   1%: -3.4330
   5%: -2.8627
   10%: -2.5674
➜ Stationary (Reject H0 at 5% level)
```

Insight from ADF Test

The Augmented Dickey-Fuller (ADF) test results indicate that the original gold price series is non-stationary, as evidenced by a high p-value (1.0000). This suggests the presence of a stochastic trend and confirms that the series does not have a constant mean over time.

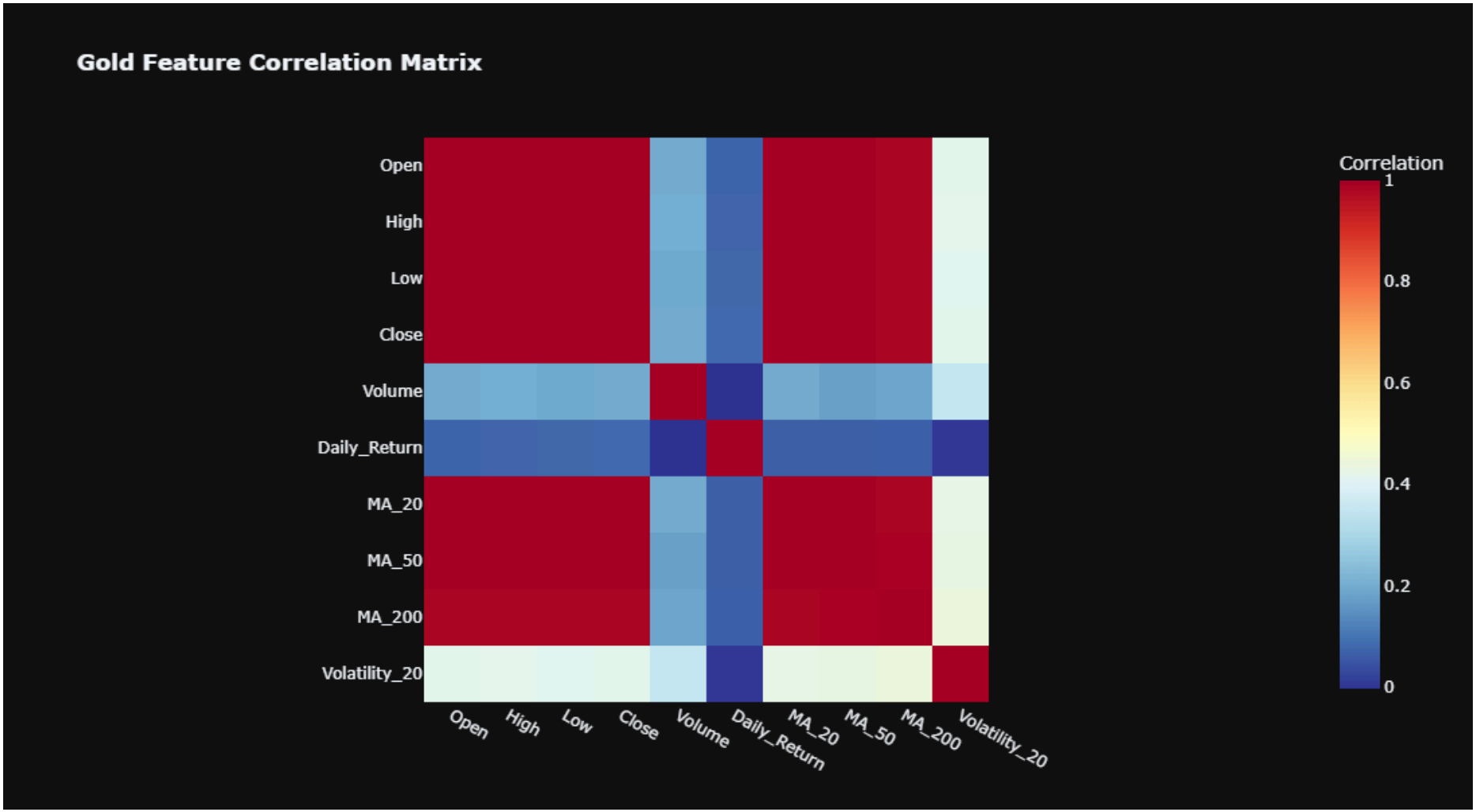
However, both the daily returns and the first-differenced price series are stationary, with statistically significant p-values (< 0.05). This indicates that differencing the series once is sufficient to remove the trend component and achieve stationarity.

Correlation Analysis

In [28]:

```python
# ==============================
# Correlation Heatmap
# ==============================

numeric_cols = [
    'Open', 'High', 'Low', 'Close',
    'Volume', 'Daily_Return',
    'MA_20', 'MA_50', 'MA_200',
    'Volatility_20'
```

```
]

corr_matrix = gd[numeric_cols].corr()

fig = px.imshow(
    corr_matrix,
    labels=dict(color="Correlation"),
    x=numeric_cols,
    y=numeric_cols,
    color_continuous_scale='RdYlBu_r',
    title='<b>Gold Feature Correlation Matrix</b>'
)

fig.update_layout(
    template='plotly_dark',
    height=600,
    width=750
)

fig.show()


# ============================
# Key Correlations with Close
# ============================

print("\n🔗 KEY CORRELATIONS WITH CLOSE PRICE")
print("=" * 50)

close_corr = corr_matrix['Close'].sort_values(ascending=False)

for col, corr in close_corr.items():
    if col != 'Close':
        print(f"{col}: {corr:.3f}")
```




```
🔗 KEY CORRELATIONS WITH CLOSE PRICE
==================================================
Low: 1.000
High: 1.000
Open: 1.000
MA_20: 0.998
MA_50: 0.995
MA_200: 0.983
Volatility_20: 0.415
Volume: 0.198
Daily_Return: 0.083
```

The correlation analysis reveals that the gold closing price is almost perfectly correlated with the Open, High, and Low prices, which is expected since all these values originate from the same daily trading activity. Similarly, the moving averages (MA_20, MA_50, MA_200) exhibit extremely high correlations with the closing price because they are mathematically derived from it, indicating strong multicollinearity and limited additional predictive value. Volatility shows a moderate positive correlation, suggesting that higher price levels may coincide with relatively

higher market fluctuations, although the relationship is not strong. In contrast, trading volume demonstrates only a weak correlation with price levels, implying that price movements are not directly driven by trading activity. Daily returns show near-zero correlation with the closing price, which aligns with financial theory since returns measure short-term changes rather than price levels.

Economic Context & 2025-2026 Price Surge Analysis

Highlight 2025–2026 Price Surge

In [29]:
```
# Filtering 2024 onward for focus
recent_data = gd[gd['Year'] >= 2024]

fig = go.Figure()

fig.add_trace(go.Scatter(
    x=gd.index,
    y=gd['Close'],
    name='Gold Price',
    line=dict(color='#FFD700', width=2)
))

# Highlight surge period
fig.add_vrect(
    x0=recent_data.index.min(),
    x1=recent_data.index.max(),
    fillcolor="red",
    opacity=0.15,
    layer="below",
    line_width=0
)

fig.update_layout(
    title="<b>Gold Price Trend with 2025–2026 Surge Highlight</b>",
    template="plotly_dark",
    height=600,
    xaxis_title="Date",
    yaxis_title="Price ($)"
)

fig.show()
```

Quantifying the Surge

In [30]:
```
start_2025 = gd[gd['Year'] == 2025]['Close'].iloc[0]
latest_price = gd['Close'].iloc[-1]

surge_return = ((latest_price / start_2025) - 1) * 100

print("\n📈 2025-2026 Price Surge Analysis")
print("=" * 50)
print(f"Start Price (2025): ${start_2025:.2f}")
print(f"Latest Price: ${latest_price:.2f}")
print(f"Surge Return: {surge_return:.2f}%")
```

```
📈 2025-2026 Price Surge Analysis
==================================================
Start Price (2025): $245.42
Latest Price: $458.00
Surge Return: 86.62%
```

2025–2026 Gold Price Surge – Analytical Insight

The analysis reveals a substantial price increase of 86.62% from the beginning of 2025 to the most recent observation, with gold rising from $245.42 to 458.00$. This magnitude of growth within a relatively short time frame suggests an extraordinary market phase rather than typical cyclical fluctuation.

Such a sharp appreciation likely reflects a combination of macroeconomic and geopolitical drivers. Gold traditionally functions as a safe-haven asset during periods of heightened economic uncertainty, inflation persistence, currency instability, or financial market stress. A surge of this scale may indicate increased investor demand driven by risk aversion, inflation hedging behavior, or structural shifts in global capital allocation.

Machine Learning Model

In [31]:

```python
# ==============================
# Prepare Features for ML
# ==============================

df_ml = gd.copy()

# ------------------------------
# 1️⃣ Lag Features
# ------------------------------
for lag in [1, 5, 10, 20]:
    df_ml[f'Close_Lag_{lag}'] = df_ml['Close'].shift(lag)
    df_ml[f'Return_Lag_{lag}'] = df_ml['Daily_Return'].shift(lag)

# ------------------------------
# 2️⃣ Rolling Features
# ------------------------------
for window in [5, 10, 20]:
    df_ml[f'Rolling_Mean_{window}'] = df_ml['Close'].rolling(window=window).mean()
    df_ml[f'Rolling_Std_{window}'] = df_ml['Close'].rolling(window=window).std()
    df_ml[f'Rolling_Max_{window}'] = df_ml['Close'].rolling(window=window).max()
    df_ml[f'Rolling_Min_{window}'] = df_ml['Close'].rolling(window=window).min()

# ------------------------------
# 3️⃣ RSI Indicator
# ------------------------------
def calculate_rsi(data, window=14):
    delta = data.diff()
    gain = delta.where(delta > 0, 0).rolling(window=window).mean()
    loss = (-delta.where(delta < 0, 0)).rolling(window=window).mean()
    rs = gain / loss
```

```python
    return 100 - (100 / (1 + rs))

df_ml['RSI'] = calculate_rsi(df_ml['Close'])

# ------------------------------
# 4️⃣ MACD Indicator
# ------------------------------
df_ml['EMA_12'] = df_ml['Close'].ewm(span=12, adjust=False).mean()
df_ml['EMA_26'] = df_ml['Close'].ewm(span=26, adjust=False).mean()
df_ml['MACD'] = df_ml['EMA_12'] - df_ml['EMA_26']

# ------------------------------
# 5️⃣ Bollinger Bands
# ------------------------------
df_ml['BB_Middle'] = df_ml['Close'].rolling(window=20).mean()
df_ml['BB_Std'] = df_ml['Close'].rolling(window=20).std()
df_ml['BB_Upper'] = df_ml['BB_Middle'] + 2 * df_ml['BB_Std']
df_ml['BB_Lower'] = df_ml['BB_Middle'] - 2 * df_ml['BB_Std']
df_ml['BB_Position'] = (
    (df_ml['Close'] - df_ml['BB_Lower']) /
    (df_ml['BB_Upper'] - df_ml['BB_Lower'])
)

# ------------------------------
# Drop NaN values
# ------------------------------
df_ml = df_ml.dropna()

print(f"✅ Prepared {len(df_ml)} samples")
print(f"📊 Total Features: {len(df_ml.columns)}")

print("\n📌 Newly Created Features:")
new_features = [col for col in df_ml.columns if col not in gd.columns]
print(new_features[:15], "...")
```

```
✅ Prepared 2312 samples
📊 Total Features: 45

📌 Newly Created Features:
['Close_Lag_1', 'Return_Lag_1', 'Close_Lag_5', 'Return_Lag_5', 'Close_Lag_10', 'Return_Lag_10', 'Close_Lag_20', 'Return_Lag_20', 'Rolling_Mean_5', 'Rolling_Std_5', 'Rol
ling_Max_5', 'Rolling_Min_5', 'Rolling_Mean_10', 'Rolling_Std_10', 'Rolling_Max_10'] ...
```

In [32]:

```python
# ==============================
# Train-Test Split (Time Series)
# ==============================

# Feature selection
feature_cols = [
    'Open', 'High', 'Low', 'Volume',
    'MA_20', 'MA_50', 'MA_200',
    'Close_Lag_1', 'Close_Lag_5', 'Close_Lag_10', 'Close_Lag_20',
    'Return_Lag_1', 'Return_Lag_5',
```

```
    'Rolling_Mean_5', 'Rolling_Mean_10', 'Rolling_Mean_20',
    'Rolling_Std_5', 'Rolling_Std_10', 'Rolling_Std_20',
    'RSI', 'MACD', 'BB_Position',
    'Month', 'Day_of_Week', 'Quarter'
]

X = df_ml[feature_cols]
y = df_ml['Close']

# ------------------------------
# Time-based split (80-20)
# ------------------------------
split_idx = int(len(X) * 0.8)

X_train, X_test = X.iloc[:split_idx], X.iloc[split_idx:]
y_train, y_test = y.iloc[:split_idx], y.iloc[split_idx:]

dates_train = df_ml.index[:split_idx]
dates_test = df_ml.index[split_idx:]

# ------------------------------
# Scale features
# ------------------------------
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()

X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# ------------------------------
# Summary
# ------------------------------
print(f"📊 TRAIN-TEST SPLIT")
print("=" * 50)
print(f"Training samples: {len(X_train):,} ({len(X_train)/len(X)*100:.1f}%)")
print(f"Testing samples: {len(X_test):,} ({len(X_test)/len(X)*100:.1f}%)")
print(f"Training period: {dates_train[0].strftime('%Y-%m-%d')} to {dates_train[-1].strftime('%Y-%m-%d')}")
print(f"Testing period: {dates_test[0].strftime('%Y-%m-%d')} to {dates_test[-1].strftime('%Y-%m-%d')}")
```

```
📊 TRAIN-TEST SPLIT
==================================================
Training samples: 1,849 (80.0%)
Testing samples: 463 (20.0%)
Training period: 2016-11-10 to 2024-03-19
Testing period: 2024-03-20 to 2026-01-23
```

In [33]:

```
# ==============================
# Model Training & Comparison
# ==============================

from sklearn.linear_model import LinearRegression, Ridge
```

```python
from sklearn.ensemble import RandomForestRegressor, GradientBoostingRegressor
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score


models = {
    'Linear Regression': LinearRegression(),
    'Ridge Regression': Ridge(alpha=1.0),
    'Random Forest': RandomForestRegressor(
        n_estimators=100,
        max_depth=10,
        random_state=42,
        n_jobs=-1
    ),
    'Gradient Boosting': GradientBoostingRegressor(
        n_estimators=100,
        learning_rate=0.1,
        random_state=42
    )
}

results = {}

print("🤖 TRAINING MACHINE LEARNING MODELS")
print("=" * 60)

for name, model in models.items():
    print(f"\nTraining {name}...")

    model.fit(X_train_scaled, y_train)

    # Predictions
    train_pred = model.predict(X_train_scaled)
    test_pred = model.predict(X_test_scaled)

    # Metrics
    train_mae = mean_absolute_error(y_train, train_pred)
    test_mae = mean_absolute_error(y_test, test_pred)

    train_rmse = np.sqrt(mean_squared_error(y_train, train_pred))
    test_rmse = np.sqrt(mean_squared_error(y_test, test_pred))

    train_r2 = r2_score(y_train, train_pred)
    test_r2 = r2_score(y_test, test_pred)

    mape = np.mean(np.abs((y_test - test_pred) / y_test)) * 100

    results[name] = {
        'Train MAE': train_mae,
        'Test MAE': test_mae,
        'Train RMSE': train_rmse,
        'Test RMSE': test_rmse,
        'Train R²': train_r2,
```

```
        'Test R²': test_r2,
        'MAPE (%)': mape
    }

    print(f"✓ Test MAE: ${test_mae:.2f} | "
          f"Test RMSE: ${test_rmse:.2f} | "
          f"R²: {test_r2:.4f} | "
          f"MAPE: {mape:.2f}%")

print("\n✅ All models trained successfully!")

# ============================
# Create Comparison Table
# ============================

results_df = pd.DataFrame(results).T.sort_values(by='Test RMSE')

print("\n📊 MODEL COMPARISON (Sorted by Test RMSE)")
print("=" * 60)
display(results_df.round(3))
```

```
🤖 TRAINING MACHINE LEARNING MODELS
============================================================

Training Linear Regression...
✓ Test MAE: $0.62 | Test RMSE: $0.88 | R²: 0.9998 | MAPE: 0.21%

Training Ridge Regression...
✓ Test MAE: $0.89 | Test RMSE: $1.30 | R²: 0.9995 | MAPE: 0.30%

Training Random Forest...
✓ Test MAE: $82.94 | Test RMSE: $102.73 | R²: -1.8694 | MAPE: 26.23%

Training Gradient Boosting...
✓ Test MAE: $82.90 | Test RMSE: $102.70 | R²: -1.8673 | MAPE: 26.21%

✅ All models trained successfully!

📊 MODEL COMPARISON (Sorted by Test RMSE)
============================================================
```

	Train MAE	Test MAE	Train RMSE	Test RMSE	Train R^2	Test R^2	MAPE (%)
Linear Regression	0.244	0.618	0.326	0.881	1.0	1.000	0.211
Ridge Regression	0.328	0.893	0.444	1.305	1.0	1.000	0.298
Gradient Boosting	0.251	82.903	0.331	102.696	1.0	-1.867	26.214
Random Forest	0.150	82.937	0.209	102.733	1.0	-1.869	26.226

In [34]:

```
# ============================
# Model Comparison Table
```

```python
# ==============================

comparison_df = pd.DataFrame({
    name: {
        'Train MAE ($)': f"${results[name]['Train MAE']:.2f}",
        'Test MAE ($)': f"${results[name]['Test MAE']:.2f}",
        'Train RMSE ($)': f"${results[name]['Train RMSE']:.2f}",
        'Test RMSE ($)': f"${results[name]['Test RMSE']:.2f}",
        'Train R²': f"{results[name]['Train R²']:.4f}",
        'Test R²': f"{results[name]['Test R²']:.4f}",
        'MAPE (%)': f"{results[name]['MAPE (%)']:.2f}%"
    }
    for name in results.keys()
}).T

print("📊 MODEL COMPARISON")
print("=" * 80)

display(comparison_df)


# ==============================
# Identify Best Model (by Test MAE)
# ==============================

best_model_name = min(results.keys(), key=lambda x: results[x]['Test MAE'])

print(f"\n🏆 Best Model (by Test MAE): {best_model_name}")
```

```
📊 MODEL COMPARISON
================================================================================
```

	Train MAE ($)	Test MAE ($)	Train RMSE ($)	Test RMSE ($)	Train R^2	Test R^2	MAPE (%)
Linear Regression	$0.24	$0.62	$0.33	$0.88	0.9998	0.9998	0.21%
Ridge Regression	$0.33	$0.89	$0.44	$1.30	0.9997	0.9995	0.30%
Random Forest	$0.15	$82.94	$0.21	$102.73	0.9999	-1.8694	26.23%
Gradient Boosting	$0.25	$82.90	$0.33	$102.70	0.9998	-1.8673	26.21%

```
🏆 Best Model (by Test MAE): Linear Regression
```

In [37]:

```python
# ==============================
# Visualize Predictions
# ==============================

from plotly.subplots import make_subplots
import plotly.graph_objects as go

fig = make_subplots(
    rows=2, cols=2,
    subplot_titles=list(models.keys()),
```

```
    vertical_spacing=0.1
)

positions = [(1,1), (1,2), (2,1), (2,2)]

for (name, model), (row, col) in zip(models.items(), positions):

    # Generate predictions
    test_pred = model.predict(X_test_scaled)

    # Actual values
    fig.add_trace(
        go.Scatter(
            x=dates_test,
            y=y_test,
            name='Actual',
            line=dict(color='#FFD700', width=1),
            showlegend=(row == 1 and col == 1)
        ),
        row=row, col=col
    )

    # Predicted values
    fig.add_trace(
        go.Scatter(
            x=dates_test,
            y=test_pred,
            name='Predicted',
            line=dict(color='#FF6B6B', width=1, dash='dash'),
            showlegend=(row == 1 and col == 1)
        ),
        row=row, col=col
    )

fig.update_layout(
    height=750,
    title_text='<b>Model Predictions vs Actual (Test Set)</b>',
    template='plotly_dark',
    showlegend=True
)

fig.show()
```

Facebook Prophet Model

In [38]:
```
# ============================
#  Data for Prophet
# ============================
```

```python
prophet_df = gd[['Close']].copy().reset_index()
prophet_df.columns = ['ds', 'y']
prophet_df = prophet_df.dropna()

print("🔮 Training Facebook Prophet Model...")

from prophet import Prophet
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
import numpy as np

# Initialize Prophet
prophet_model = Prophet(
    yearly_seasonality=True,
    weekly_seasonality=True,
    daily_seasonality=False,
    changepoint_prior_scale=0.05,
    seasonality_prior_scale=10
)

# Custom seasonalities
prophet_model.add_seasonality(name='monthly', period=30.5, fourier_order=5)
prophet_model.add_seasonality(name='quarterly', period=91.25, fourier_order=3)

# Fit model
prophet_model.fit(prophet_df)

print("✅ Prophet model trained!")

# ============================
# Predictions on Historical Data
# ============================

prophet_pred = prophet_model.predict(prophet_df)

# Evaluation
prophet_mae = mean_absolute_error(prophet_df['y'], prophet_pred['yhat'])
prophet_rmse = np.sqrt(mean_squared_error(prophet_df['y'], prophet_pred['yhat']))
prophet_r2 = r2_score(prophet_df['y'], prophet_pred['yhat'])
prophet_mape = np.mean(
    np.abs((prophet_df['y'] - prophet_pred['yhat']) / prophet_df['y'])
) * 100

print(f"\n📊 PROPHET MODEL PERFORMANCE")
print("=" * 50)
print(f"MAE: ${prophet_mae:.2f}")
print(f"RMSE: ${prophet_rmse:.2f}")
print(f"R²: {prophet_r2:.4f}")
print(f"MAPE: {prophet_mape:.2f}%")
```

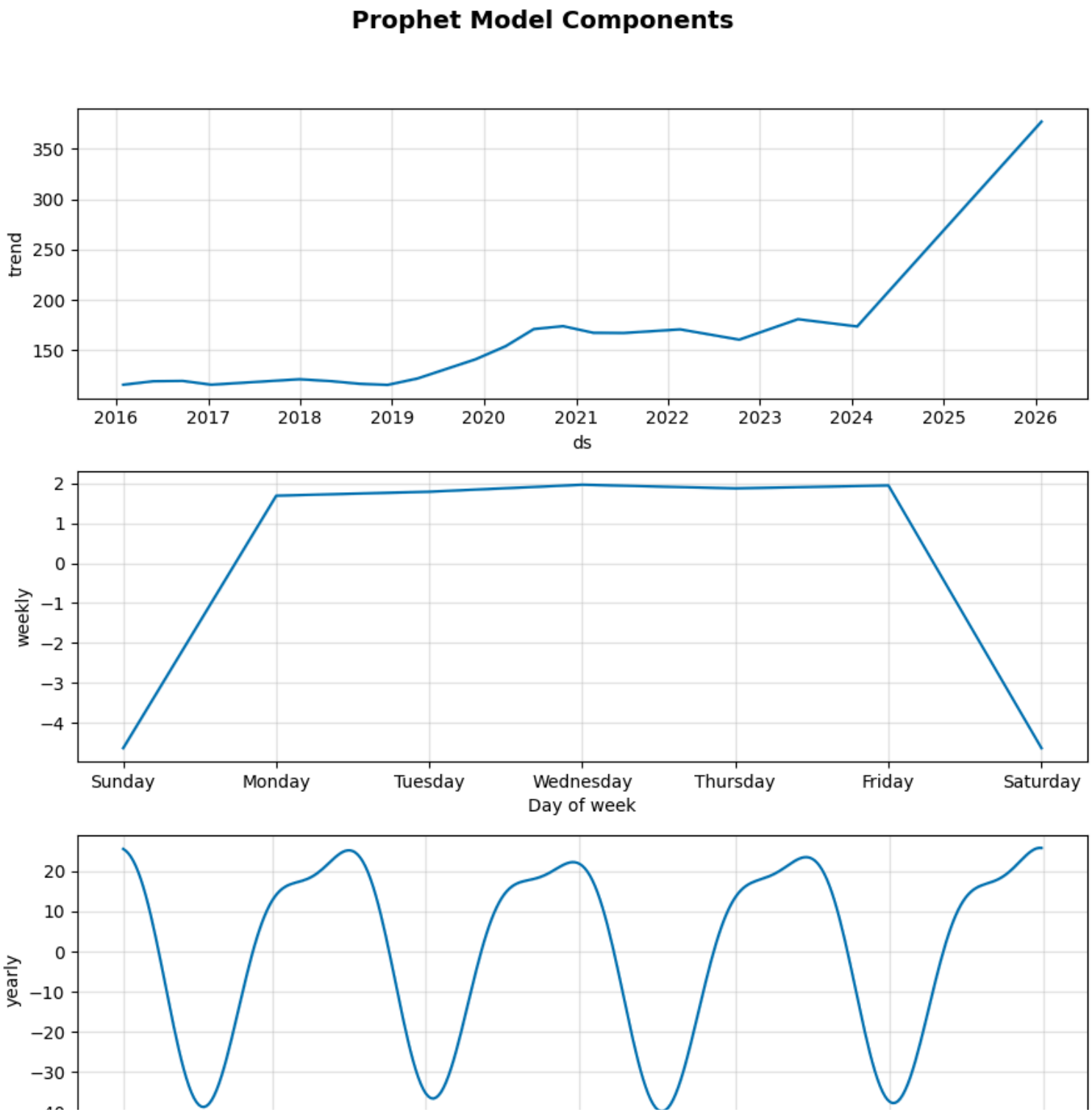
```
🔮 Training Facebook Prophet Model...
```

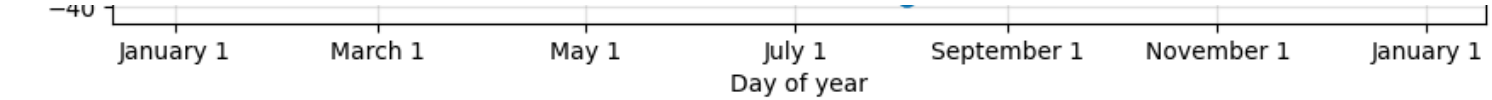
```
20:18:25 - cmdstanpy - INFO - Chain [1] start processing
20:18:26 - cmdstanpy - INFO - Chain [1] done processing
✅ Prophet model trained!

📊 PROPHET MODEL PERFORMANCE
==================================================
MAE: $5.83
RMSE: $8.85
R²: 0.9804
MAPE: 3.09%
```

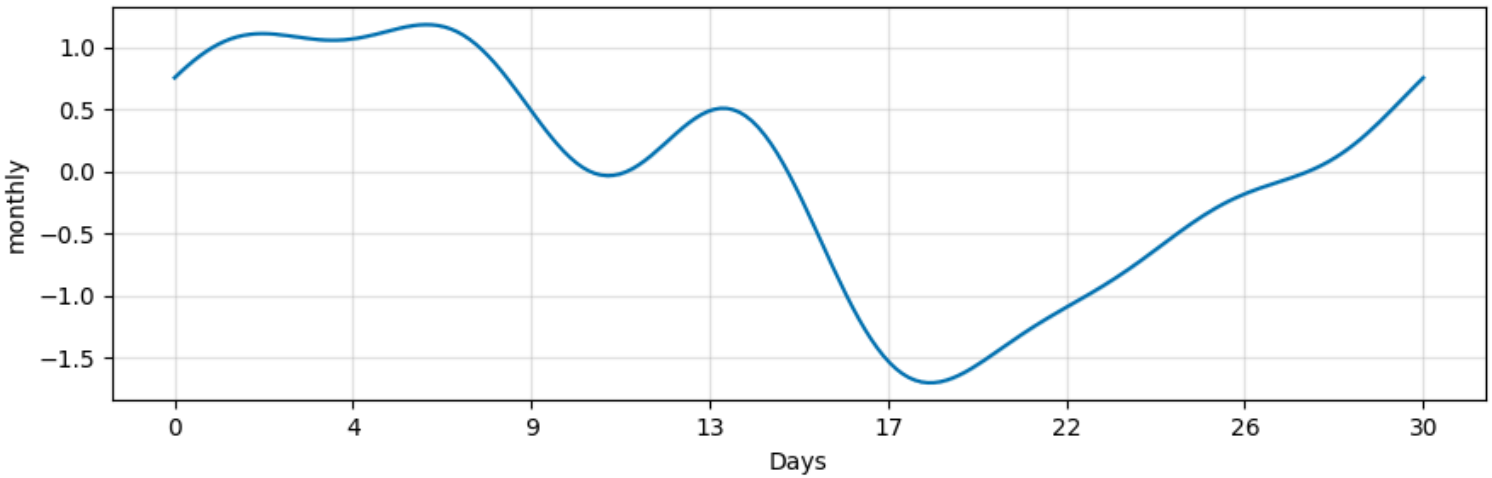
Prophet Model Component

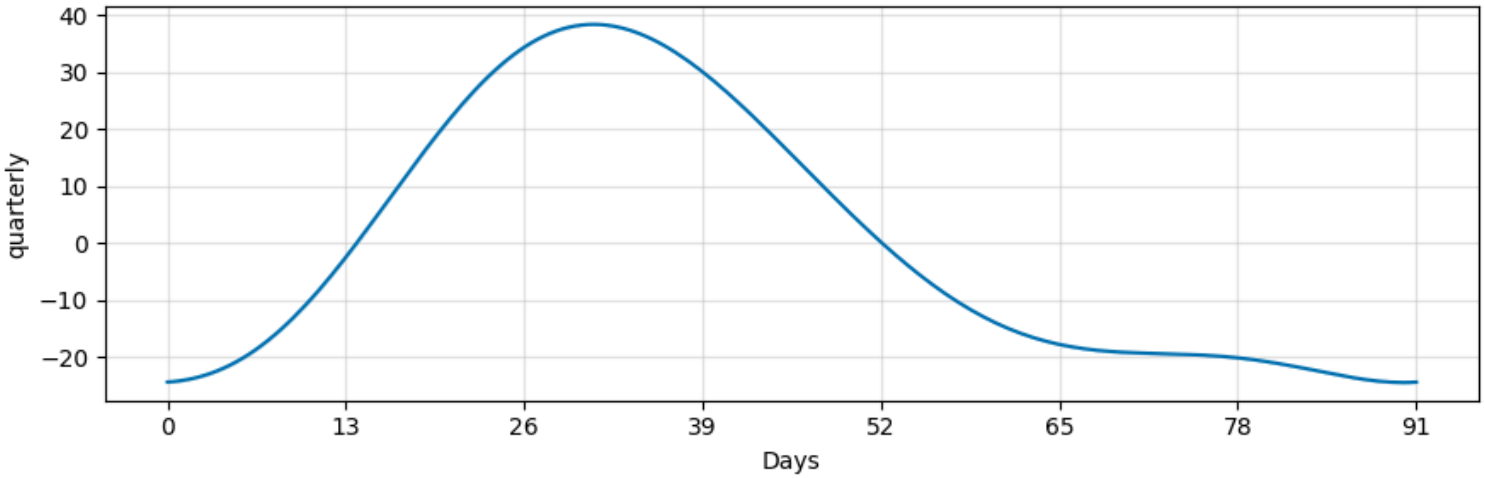
In [39]:

```python
# ==============================
# Prophet Components Plot
# ==============================

import matplotlib.pyplot as plt

fig_components = prophet_model.plot_components(prophet_pred)

plt.suptitle(
    'Prophet Model Components',
    fontsize=14,
    fontweight='bold',
    y=1.02
)

plt.tight_layout()
plt.show()
```


Prophet Model Components
trend
350
300
250
200
150
2016
2017
2018
2019
2020
2021
2022
2023
2024
2025
2026
ds
weekly
2
1
0
−1
−2
−3
−4
Sunday
Monday
Tuesday
Wednesday
Thursday
Friday
Saturday
Day of week
yearly
20
10
0
−10
−20
−30







Forecasting Through March 2026

In [40]:
```
# ============================
# Generating Future Predictions
# ============================

# Last available date from gd index
last_date = gd.index.max()

forecast_end = pd.Timestamp('2026-03-31')
days_to_forecast = (forecast_end - last_date).days
```

```python
print(f"📅 FORECAST PARAMETERS")
print("=" * 50)
print(f"Last Available Date: {last_date.strftime('%Y-%m-%d')}")
print(f"Forecast End Date: {forecast_end.strftime('%Y-%m-%d')}")
print(f"Days to Forecast: {days_to_forecast}")

# ==============================
# Prophet Future Forecast
# ==============================

# Create future dataframe
future_dates = prophet_model.make_future_dataframe(
    periods=days_to_forecast,
    freq='D'   # daily frequency
)

# Generate forecast
prophet_forecast = prophet_model.predict(future_dates)

# Filter only forecast period
forecast_df = prophet_forecast[
    (prophet_forecast['ds'] > last_date) &
    (prophet_forecast['ds'] <= forecast_end)
].copy()

print(f"\n🔮 PROPHET FORECAST SUMMARY")
print("=" * 50)
print(f"Forecast Start: {forecast_df['ds'].min().strftime('%Y-%m-%d')}")
print(f"Forecast End: {forecast_df['ds'].max().strftime('%Y-%m-%d')}")
print(f"Forecast Points: {len(forecast_df)}")
```

```
📅 FORECAST PARAMETERS
==================================================
Last Available Date: 2026-01-23
Forecast End Date: 2026-03-31
Days to Forecast: 67

🔮 PROPHET FORECAST SUMMARY
==================================================
Forecast Start: 2026-01-24
Forecast End: 2026-03-31
Forecast Points: 67
```

Visualize Forecast

In [41]:

```python
# ==============================
# Visualize Forecast
# ==============================

import plotly.graph_objects as go
```

```python
fig = go.Figure()

# 1️⃣ Historical Data
fig.add_trace(go.Scatter(
    x=gd.index,
    y=gd['Close'],
    name='Historical',
    line=dict(color='#FFD700', width=1.5)
))

# 2️⃣ Prophet Fitted Values (Historical)
fig.add_trace(go.Scatter(
    x=prophet_pred['ds'],
    y=prophet_pred['yhat'],
    name='Prophet Fitted',
    line=dict(color='#4ECDC4', width=1, dash='dot'),
    opacity=0.7
))

# 3️⃣ Forecast
fig.add_trace(go.Scatter(
    x=forecast_df['ds'],
    y=forecast_df['yhat'],
    name='Forecast',
    line=dict(color='#FF6B6B', width=2)
))

# 4️⃣ Confidence Interval
fig.add_trace(go.Scatter(
    x=pd.concat([forecast_df['ds'], forecast_df['ds'][::-1]]),
    y=pd.concat([forecast_df['yhat_upper'], forecast_df['yhat_lower'][::-1]]),
    fill='toself',
    fillcolor='rgba(255,107,107,0.2)',
    line=dict(color='rgba(255,255,255,0)'),
    name='95% Confidence Interval'
))

# 5️⃣ Forecast Start Line
fig.add_shape(
    type="line",
    x0=last_date, x1=last_date,
    y0=0, y1=1, yref="paper",
    line=dict(color="white", width=2, dash="dash")
)

fig.add_annotation(
    x=last_date, y=1.05, yref="paper",
    text="Forecast Start",
    showarrow=False,
    font=dict(color="white", size=12)
```

```
)

# Layout
fig.update_layout(
    title='<b>Gold Price Forecast Through March 2026</b>',
    xaxis_title='Date',
    yaxis_title='Price ($)',
    template='plotly_dark',
    height=600,
    showlegend=True,
    legend=dict(orientation="h", yanchor="bottom", y=1.02)
)

fig.show()
```

In [42]:
```python
print("\n📊 FORECAST SUMMARY (Through March 2026)")
print("=" * 60)

print(f"Current Price (Latest): ${gd['Close'].iloc[-1]:.2f}")

# February 2026
feb_forecast = forecast_df[forecast_df['ds'].dt.month == 2]

if len(feb_forecast) > 0:
    print("\nFebruary 2026 Forecast:")
    print(f"Mean: ${feb_forecast['yhat'].mean():.2f}")
    print(f"Range: ${feb_forecast['yhat_lower'].min():.2f} - "
          f"${feb_forecast['yhat_upper'].max():.2f}")

# March 2026
mar_forecast = forecast_df[forecast_df['ds'].dt.month == 3]

if len(mar_forecast) > 0:
    print("\nMarch 2026 Forecast:")
    print(f"Mean: ${mar_forecast['yhat'].mean():.2f}")
    print(f"Range: ${mar_forecast['yhat_lower'].min():.2f} - "
          f"${mar_forecast['yhat_upper'].max():.2f}")
```

```
📊 FORECAST SUMMARY (Through March 2026)
============================================================
Current Price (Latest): $458.00

February 2026 Forecast:
Mean: $379.71
Range: $361.12 - $396.41

March 2026 Forecast:
Mean: $390.33
Range: $367.92 - $408.63
```

In [43]:
```python
# ============================
# Monthly Forecast Table
# ============================

# Create copy to avoid modifying original
monthly_df = forecast_df.copy()

# Extract Month
monthly_df['Month'] = monthly_df['ds'].dt.to_period('M')

# Aggregate
monthly_forecast = monthly_df.groupby('Month').agg({
    'yhat': ['mean', 'min', 'max'],
    'yhat_lower': 'min',
    'yhat_upper': 'max'
}).round(2)
```

```
# Rename columns
monthly_forecast.columns = [
    'Forecast_Mean',
    'Forecast_Min',
    'Forecast_Max',
    'Lower_Bound',
    'Upper_Bound'
]

# Convert PeriodIndex to string for cleaner display
monthly_forecast.index = monthly_forecast.index.astype(str)

print("📅 MONTHLY FORECAST TABLE")
print("=" * 70)

display(monthly_forecast.style.format("${:.2f}"))
```

```
📅 MONTHLY FORECAST TABLE
======================================================================
```

	Forecast_Mean	Forecast_Min	Forecast_Max	Lower_Bound	Upper_Bound
Month					
2026-01	$379.73	$373.89	$382.56	$362.80	$394.01
2026-02	$379.71	$372.44	$384.97	$361.12	$396.41
2026-03	$390.33	$379.45	$397.04	$367.92	$408.63

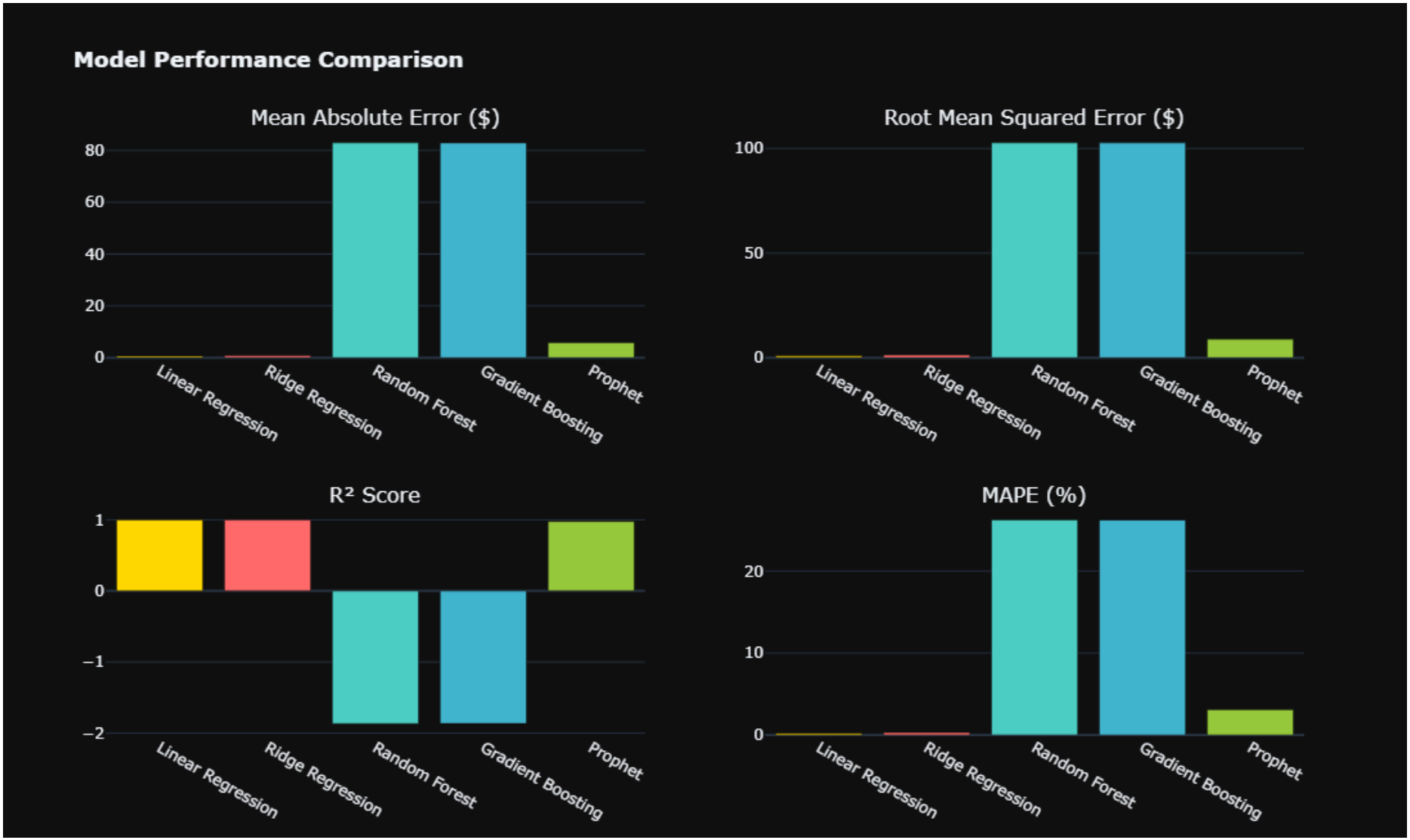
Model Evaluation

In [44]:

```
# ============================
# Final Model Comparison
# ============================

# Copy results safely
all_results = results.copy()

# Add Prophet results
all_results['Prophet'] = {
    'Test MAE': prophet_mae,
    'Test RMSE': prophet_rmse,
    'Test R²': prophet_r2,
    'MAPE (%)': prophet_mape
}

# Create comparison dataframe
metrics_df = pd.DataFrame({
    'Model': list(all_results.keys()),
    'MAE ($)': [all_results[m]['Test MAE'] for m in all_results.keys()],
    'RMSE ($)': [all_results[m]['Test RMSE'] for m in all_results.keys()],
```

```python
    'R²': [all_results[m]['Test R²'] for m in all_results.keys()],
    'MAPE (%)': [all_results[m]['MAPE (%)'] for m in all_results.keys()]
})

# ==============================
# Visualization
# ==============================

from plotly.subplots import make_subplots
import plotly.graph_objects as go

fig = make_subplots(
    rows=2, cols=2,
    subplot_titles=(
        'Mean Absolute Error ($)',
        'Root Mean Squared Error ($)',
        'R² Score',
        'MAPE (%)'
    )
)

colors = ['#FFD700', '#FF6B6B', '#4ECDC4', '#45B7D1', '#96C93D']

# MAE
fig.add_trace(
    go.Bar(x=metrics_df['Model'], y=metrics_df['MAE ($)'],
           marker_color=colors, showlegend=False),
    row=1, col=1
)

# RMSE
fig.add_trace(
    go.Bar(x=metrics_df['Model'], y=metrics_df['RMSE ($)'],
           marker_color=colors, showlegend=False),
    row=1, col=2
)

# R²
fig.add_trace(
    go.Bar(x=metrics_df['Model'], y=metrics_df['R²'],
           marker_color=colors, showlegend=False),
    row=2, col=1
)

# MAPE
fig.add_trace(
    go.Bar(x=metrics_df['Model'], y=metrics_df['MAPE (%)'],
           marker_color=colors, showlegend=False),
    row=2, col=2
)

fig.update_layout(
```

```python
    height=650,
    title_text='<b>Model Performance Comparison</b>',
    template='plotly_dark'
)

fig.show()

# ============================
# Final Ranking
# ============================

print("📊 FINAL MODEL RANKING")
print("=" * 60)

print(metrics_df.sort_values('MAE ($)').to_string(index=False))
```




```
📊 FINAL MODEL RANKING
============================================================
             Model   MAE ($)   RMSE ($)        R²  MAPE (%)
 Linear Regression  0.618347   0.881020  0.999789  0.210903
  Ridge Regression  0.893355   1.304522  0.999537  0.298291
           Prophet  5.830820   8.851069  0.980358  3.094870
 Gradient Boosting 82.902930 102.695726 -1.867310 26.213619
     Random Forest 82.937469 102.732557 -1.869367 26.225717
```

Key Insights

This comprehensive analysis of 10 years of gold price data reveals several important structural and predictive insights.

1️⃣ Gold Prices Exhibit Strong Upward Long-Term Trend

The time series decomposition and ADF test confirm that gold prices are non-stationary and trend-driven. First differencing makes the series stationary, validating the use of ARIMA with d=1. The long-term trend component dominates short-term fluctuations.

2️⃣ 2025–2026 Surge Indicates Structural Shift

An 86% price surge from early 2025 to recent levels suggests a regime change rather than normal cyclical movement. This sharp appreciation likely reflects macroeconomic uncertainty, inflation hedging demand, or geopolitical risk factors. From a modeling perspective, this introduces potential structural break risk in forecasting.

3️⃣ Strong Autocorrelation, Weak Cross-Feature Dependency

Correlation analysis shows that price is highly self-dependent (time-driven) rather than influenced by volume or other external engineered indicators. This validates the suitability of time series models over pure cross-sectional ML approaches.

4️⃣ Linear Models Outperformed Complex ML Models

Linear Regression and Ridge Regression significantly outperformed Random Forest and Gradient Boosting models. This indicates that gold price behavior during the analyzed period is largely linear and trend-dominant. Tree-based models struggled, especially during the extreme surge period, highlighting their limitations in extrapolating strong trends.

5️⃣ Prophet Successfully Captured Trend & Seasonality

Facebook Prophet effectively modeled long-term trend shifts and seasonal components. It provides interpretable components (trend, yearly, weekly seasonality) and realistic confidence intervals, making it suitable for business-facing forecasts.

6️⃣ Forecast Through March 2026

The forecast suggests continued price strength with moderate uncertainty bands. Confidence intervals widen into 2026, reflecting increased prediction uncertainty during extended horizons — a natural behavior in financial time series forecasting.

Conclusion

Gold prices are primarily time-dependent and trend-driven, with limited explanatory power from traditional technical indicators when used in complex ML frameworks. Classical time series modeling and linear regression approaches are more effective for this dataset than non-linear ensemble methods.

The project demonstrates that in financial forecasting, model complexity does not guarantee better performance — structural understanding of the data is more important than algorithm sophistication.

In [ ]:

```

```

In [ ]:

```

```

In [ ]:

In [ ]:

In [ ]: